LLM-augmented NIDS via MCP: a literature-backed research report

Large language models show significant potential for explainable network intrusion detection but remain fundamentally unsuitable as standalone classifiers — a finding your experimental results directly confirm and that a growing body of literature now supports. Your 83.3% baseline F1 from flow features alone, the catastrophic recall collapse in Iteration 3, and the universal failure of external tool calls each map onto documented phenomena in the academic literature: LLM overconfidence bias, multi-objective prompt interference, and CICIDS dataset limitations. The path forward, supported by both academic and industry evidence, lies in multi-agent architectures that separate statistical detection from LLM-driven explanation and temporal reasoning — a design pattern already validated by CrowdStrike Charlotte AI, Microsoft Security Copilot, and recent academic frameworks.

---

1. Literature review: LLMs applied to network intrusion detection

LLMs are poor classifiers but strong explainers

The foundational result in this space comes from your own institution. Houssel et al. (2024), working under Portmann at UQ, evaluated GPT-4 and LLaMA3-8B-Instruct for zero-shot NetFlow classification on NF-UNSW-NB15-v2 and **NF-CSE-CIC-IDS2018-v2** — the same dataset family you use. Their central conclusion: **LLMs are "unfit for the detection of malicious NetFlows"** but show "significant potential as complementary agents in NIDS, particularly in providing explanations and aiding in threat response when integrated with RAG and function calling capabilities" (arXiv) (arXiv) [1]. LLM inference was approximately **7,000× slower** than traditional classifiers (Houssel) (ResearchGate) (14,000 μs vs. <30 μs for Random Forest), making standalone deployment impractical even if accuracy were adequate. (The Moonlight) Their follow-up work, eX-NIDS (Houssel et al., 2025), formalised this insight into a framework where LLMs serve exclusively as explainers, with augmented prompts incorporating Cyber Threat Intelligence improving explanation quality by over 20% compared to basic prompts (arXiv +2) [2].

This classifier-vs-explainer distinction recurs across the literature. Ali and Kostakos (2023) took the same approach with HuntGPT, pairing a Random Forest classifier on KDD99 with GPT-3.5 Turbo as a conversational explainer integrated through SHAP and LIME. (ResearchGate +2) The LLM never classifies traffic — it translates XAI outputs into actionable analyst-readable reports, achieving **72–82.5% success rates** on CISM certification exam questions about the detected threats (arXiv) (arXiv) [3]. The XG-NID system (2024) similarly combines a heterogeneous graph neural network with LLaMA 3-8B for zero-shot explanation generation from Integrated Gradients feature importance on CICIDS2017 (arXiv) [4].

Direct classification attempts and their limits

Where researchers have used LLMs as direct classifiers, results are mixed. Guastalla et al. (2024) tested GPT-3.5, GPT-4, and Ada on CICIDS2017 for DDoS detection, (Springer) (arXiv) finding that **few-shot learning with 10 examples achieved ~87.6% accuracy**, while fine-tuning with 70 samples reached ~95% (Springer) [5]. These numbers appear strong but apply only to binary DDoS detection — a simpler task than multi-class NetFlow classification. On unlabelled data with few-shot prompting, performance "plateaus around 0.5–0.55 (near random)" (Usc) [5].

Davey et al. (2025) fine-tuned LLMs on comprehensive NetFlow datasets and reported reductions in false positives and improved novel attack detection, though their work focused on fine-tuned rather than zero-shot approaches (Springer) [6]. Lira et al. (2024) evaluated GPT-3.5, GPT-4, and an adaptation called BERTIDS on NSL-KDD, (Springer) with the BERT-based model achieving **98.01% accuracy, 98.31% precision, 98.01% recall, and 98.09% F1** (MDPI) [7] — numbers that reflect the relative simplicity of NSL-KDD rather than a breakthrough in LLM classification.

A critical 2025 study, "From Flows to Words," systematically tested zero-shot, instruction-guided, and calibrated few-shot LLMs on UNSW-NB15. **Zero-shot prompting caused multiple models (TinyLlama-1.1B, Mistral-7B) to collapse to a single class, yielding F1(+)=0** due to zero positive predictions (arXiv) [8]. The authors concluded that "traditional tabular baselines remain more stable and faster, yet the prompt-only pipeline requires no gradient training, produces human readable artifacts, and adapts easily through instructions" (arXiv) [8].

The agentic approach: tools and reasoning loops

The IDS-Agent (Li et al., 2024), presented at NeurIPS 2024 Workshop on Open-World Agents, represents the most architecturally similar system to yours. It pairs an LLM with iterative reasoning, knowledge retrieval, and classification tools for IoT network traffic. (OpenReview) Without tools, **vanilla GPT-4o achieved only 0.20 recall for attacks**. With the agentic framework, recall rose to 0.80, (OpenReview) with F1 scores of **0.97 on ACI-IoT and 0.75 on CIC-IoT** (OpenReview) [9]. Critically, the authors noted the agent's "heightened sensitivity to high-threat attacks, leading it to classify a sample as an attack even with less than 50% voting" (OpenReview) — directly mirroring your suspicious-by-default finding [9].

Bui et al. (2024), published in ACM CoNEXT, provided the most rigorous comparison across prompting, RAG, and fine-tuning approaches for intrusion detection (ACM Digital Library) (Polito) using proprietary commercial deployment data, avoiding CICIDS biases entirely [10]. Adjewa et al. (2024) proposed a hybrid framework using BERT embeddings with GMM clustering for unknown attack identification on CSE-CIC-IDS2018, (arXiv) achieving **95.6% classification accuracy and F1** after incremental learning (arXiv) [11].

Survey-level context

Three major surveys contextualise this work. Zhang et al. (2024) systematically reviewed 300+ works across 25 LLMs and 10+ downstream cybersecurity scenarios (Springer) [12]. Ferrag et al. (2024) provided a comprehensive review of LLM applications and vulnerabilities across intrusion detection, malware detection, and phishing (Frontiers +4) [13]. A 2025 survey in arXiv proposed a taxonomy classifying LLMs in NIDS as: (1) LLM-enhanced Processor, (2) LLM-based Detector, (3) LLM-driven Explainer, and (4) LLM-centered Controller — marking the transition from "Intelligent NIDS" to "Cognitive NIDS" (arXiv) [14].

---

2. Why the results happened: literature-backed analysis

The suspicious-by-default bias is a documented LLM property

Your baseline achieved **93.0% recall but only 13.3% benign accuracy** — the LLM classified nearly everything as malicious. This is not an artefact of your prompt design; it is a documented property of LLM-

based classification in security contexts. A 2025 study on human-AI collaboration for cybersecurity explicitly identified "the LLM's conservative bias (treating samples as suspicious unless proven otherwise)" and found it disproportionately boosted recall in intrusion detection tasks (arXiv) [15]. The IDS-Agent similarly observed that LLMs classify samples "as an attack even with less than 50% voting" (OpenReview) (OpenReview) [9].

The root cause is well-understood: **RLHF alignment training degrades calibration**. A 2026 study in BioData Mining demonstrated that pre-trained models are well-calibrated but unhelpful, while alignment makes them useful but systematically overconfident — "Human raters commonly prefer answers that sound clear and decisive. Hedged or uncertain language is often scored as less helpful" (Springer) [16]. This creates what Guo et al. (2017) first documented as the calibration problem in modern neural networks: increased model capacity leads to overconfidence (Scribd) (Semantic Scholar) [17]. For security classification specifically, the LLM has likely absorbed training signal that erring on the side of caution (flagging threats) is preferable to missing attacks — a rational prior for a security assistant that becomes pathological when used as a binary classifier.

Prompt interference explains the Iteration 3 collapse

Your most striking result is that **combining temporal memory and benign calibration (Iteration 3) produced 41.4% accuracy** — dramatically worse than either modification alone. This is a direct instance of multi-objective prompt interference. The J6 framework (2025) demonstrated that accuracy and confidence objectives in prompt optimisation "often conflict — improving accuracy may reduce confidence, and vice versa" and that "traditional prompt tuning with fixed role assignments often fails to capture the dynamic interactions between these two tasks" (arXiv) [18].

A complementary explanation comes from the "context-as-noise" effect documented in a 2026 WWW paper: "medium-level context yields consistent improvements... In contrast, adding high-level narrative and context information often reduces accuracy, sometimes performing no better than the zero-context baseline" because "high-context prompts introduce semantically rich but temporally irrelevant details that can mislead the model's internal reasoning" (arXiv) [19]. Your temporal memory and benign calibration instructions represent two additional context layers that, when combined, likely created conflicting decision pressures — the temporal context suggesting "this IP has been seen before, so normalise" while the benign calibration instruction demands "require stronger evidence before flagging." The Instruction Hierarchy paper from OpenAI (2024) further showed that LLMs "treat all input text types equally, even when messages conflict," causing performance degradation under contradictory instructions (arXiv) [20].

The specific mechanism of failure — 11 true positives suppressed versus only 6 correct benign conversions via the confidence override — demonstrates that the benign calibration instruction was applied more aggressively to borderline malicious flows than to genuinely benign ones, consistent with the finding that LLMs' "self-assessments are generally less sensitive to task difficulty than human estimates" (OpenReview) [21].

Temporal reasoning is an inherent LLM weakness

Your Iteration 1 dramatically improved benign accuracy (13.3% → 66.7%) but introduced attack normalisation — the LLM learned to treat repeated IP appearances as evidence of legitimacy. This reflects a fundamental limitation: **LLMs process interactions independently and lack native state maintenance**. The SagaLLM framework (VLDB 2025) documented that "LLMs process each interaction independently, lacking native mechanisms to maintain the state across sequential interactions" and "inherently lack robust self-validation mechanisms" (VLDB Endowment) [22]. Temporal reasoning specifically is among the weakest LLM capabilities;

the Context Rot study (2025) found that "in non-thinking mode, models perform best on knowledge-update, then multi-session, then temporal reasoning — temporal reasoning is consistently weakest" (Trychroma) [23].

Zero meaningful external tool returns reflect dataset limitations, not architecture failure

The fact that **0 of 142 external tool calls returned meaningful data** is entirely predictable given CICIDS2018's construction. All source IPs are either RFC 1918 private addresses or expired 2018 AWS IPs. AbuseIPDB, AlienVault OTX, and geolocation services have no records for private addresses by definition. This is not a failure of your MCP tool integration — it is a fundamental limitation of using lab-generated datasets with non-routable addresses for a system designed to query real-world threat intelligence. The MCP architecture would function as designed against production traffic with public IPs.

---

3. CICIDS dataset limitations: what the literature says

Systematic errors in flow construction and labelling

Engelen et al. (2021) conducted the definitive critique of CICIDS2017 (IEEE Xplore) and, by extension, CSE-CIC-IDS2018. (Distrinet-research) Their analysis found that **more than 20% of original traffic traces required reconstruction or relabelling** (Distrinet-research) [24]. The core technical problem is CICFlowMeter's incorrect flow termination: it "incorrectly terminates TCP flows after a single FIN packet" rather than requiring a mutual FIN exchange, creating erroneous flows and inverting source/destination directions (Distrinet-research) (HAL) [24]. Lanvin et al. (2023) extended these findings, documenting packet duplication and misordering in traffic captures that cause "source/destination inversion and wrong feature values," (ACM Digital Library) (ACM Digital Library) concluding that "results on the original dataset look like overfitting" (HAL) [25]. Liu et al. (2022) confirmed these errors persist in CSE-CIC-IDS2018, the dataset underlying your NetFlow V3 data (ACM Digital Library) [26].

Class imbalance and representativeness problems

Panigrahi and Borah (2018) documented severe class imbalance in CICIDS2017: (ResearchGate) BENIGN contains 2,359,087 instances while Heartbleed has only 11 and Infiltration only 36 (GitHub) — "issues sufficient enough to bias the detection engine of any typical IDS" (ResearchGate) [27]. Leevy and Khoshgoftaar (2020) found that CSE-CIC-IDS2018 contains ~16 million instances with ~17% attack traffic, (Emergent Mind) and that "best performance scores for each study were unexpectedly high overall, which may be due to overfitting" (SpringerOpen) [28].

The generalisability problem is particularly acute. A transferability study demonstrated that "perfect detection figures obtained in the context of a public dataset may not transfer in practice" when CICIDS2017-trained models were tested against unseen but closely related traffic (ACM Digital Library) [29]. Kenyon et al. (2020) surveyed 27+ public intrusion detection datasets and found that older datasets have "limited value: often heavily filtered and anonymised, with unrealistic event distributions, and opaque design methodology" (ResearchGate) [30].

Your stratified 58-flow sample and temporal construction

Your experimental design uses 58 stratified flows — a necessarily small sample for LLM evaluation given cost and latency constraints. The NetFlow V3 dataset you use (Luay et al., 2025) adds 10 temporal features to the V2

standard, totalling 53 features Uq +2 [31]. This is the most feature-rich NetFlow representation available for CICIDS2018, but it inherits the underlying flow construction errors documented by Engelen et al. UNSW-NB15 (Moustafa and Slay, 2015) remains the strongest alternative, with modern attack types and more realistic normal traffic, IEEE Xplore though it predates the temporal features your system leverages [32].

---

4. Multi-agent architectures for NIDS: what works

Academic evidence for multi-agent superiority

The strongest direct evidence comes from the MALCDF framework (2025), a SOC-style distributed multi-agent system using LLaMA 3.3 70B agents with a Secure Communication Layer. Compared against both ML-IDS baselines and single-LLM approaches, **multi-agent coordination raised detection quality with acceptable latency overhead (~1.1s versus single-LLM)** and maintained consensus confidence near 0.70 [33]. A 2025 IEEE paper demonstrated a three-agent system (email verification, log analysis, IP scanning) achieving **93.6% threat detection accuracy, 87% multi-agent correlation accuracy, and 41.3% false positive reduction** versus traditional approaches on CIC-IDS 2017 [34].

AgenticCyber (2025) deployed specialised agents (Log Agent, Vision Agent, Audio Agent, Orchestrator, Responder) on Google Gemini with LangChain orchestration, achieving **96.2% F1, 420ms response latency, 65% MTTR reduction,** arXiv **and 42% false positive reduction** arXiv versus unimodal baselines [35]. Audit-LLM (2024) was the first to employ multi-agent collaboration specifically for security log analysis, leveraging LLMs' commonsense knowledge in a zero-shot manner without fine-tuning [36].

Collaboration protocols and disagreement resolution

Tran et al. (2025) surveyed multi-agent collaboration mechanisms across five dimensions, identifying three core patterns: **competition (debate + majority voting), cooperation, and coordination** — with most effective systems combining all three arXiv [37]. Quan et al. (2024) compared centralised, decentralised, and hybrid team configurations for LLM agents in incident response using the Backdoors & Breaches framework, finding that **team structure critically optimises performance** arXiv — an important consideration for your architecture design [38].

The general multi-agent survey by Guo et al. (IJCAI 2024) categorised LLM multi-agent systems IJCAI into frameworks for orchestration, problem solving, world simulation, and benchmarking, providing canonical design patterns arxiv [39]. Li et al. (2024) synthesised a five-component structure: profile, perception, self-action, mutual interaction, and evolution [40]. Talebirad and Nadiri (2023) demonstrated that dynamic multi-agent structures where agents can be added or removed arXiv handle complex tasks more efficiently than static configurations arXiv arXiv [41].

Industry validation at scale

CrowdStrike Charlotte AI operates 12+ specialised AI agents, each fine-tuned for different tasks (data retrieval, script generation, threat intelligence analysis, validation). A router agent delegates subtasks, and a validation agent reviews outputs before presentation Exabeam — achieving **75% faster answers versus**

manual methods, ~200 hours/month in automated triage, and 98%+ accuracy matching expert decisions (CrowdStrike) [42].

Microsoft Security Copilot now supports 77+ agents across Defender, Entra, Intune, and Purview, with serial, concurrent, and hybrid ("magentic") orchestration patterns. (arXiv) Critically, it supports both **MCP for tool/data access and A2A (Agent2Agent) protocols** for cross-platform agent integration. SOC analysts using the Phishing Triage Agent detected malicious emails **550% faster** [43]. Both systems validate the orchestrator-worker pattern with specialised agents — the exact architecture your results suggest is necessary.

Framework implementations

AutoGen (Wu et al., 2023, Microsoft Research) provides the most mature open-source multi-agent framework (arXiv) with 851+ citations, supporting customisable conversable agents, (ResearchGate) ReAct patterns, Docker-based code execution, (The New Stack) and native MCP support in v0.4 (Microsoft) [44]. NVISO's cyber-security-llm-agents project demonstrates AutoGen applied specifically to cybersecurity with modular EDR detection and task coordination agents. (GitHub) LangGraph implementations for security include supervisor-style multi-agent pipelines with risk-score-based escalation. (Medium)

---

5. Proposed multi-agent architecture: evidence-based design

Based on the experimental results and literature, the following five-agent architecture addresses each documented limitation of the single-agent approach. Every design decision maps to a specific paper or finding.

Component 1: Statistical pre-filter (ML classifier)

What: A lightweight Random Forest or XGBoost classifier operating on the 53 NetFlow V3 features, producing a binary classification (malicious/benign) with a calibrated probability score.

Why: Houssel et al. (2024) demonstrated that traditional classifiers achieve inference in <30 μs versus 14,000 μs for LLMs (arXiv +2) [1]. HuntGPT established the pattern of ML-first, LLM-second (ResearchGate +2) [3]. Ferrag et al. (2023) showed SecurityBERT achieving 98.2% accuracy with only 11M parameters (Semantic Scholar) [45]. The pre-filter eliminates the LLM's documented classification weakness while providing the probabilistic threshold that your confidence override in Iteration 3 attempted but lacked.

Design detail: Flows scoring >0.9 malicious probability pass directly to the explainability agent. Flows scoring <0.1 are classified benign. Flows in the **0.1–0.9 uncertainty band** — where LLM reasoning adds value — route to the Flow Analyst Agent. This addresses the precision-recall tradeoff by removing the LLM from decisions where statistical signals are already decisive.

Component 2: Flow Analyst Agent (LLM)

What: A Claude-based agent receiving only uncertain flows, equipped with MCP tools for AbuseIPDB, AlienVault OTX, and MITRE ATT&CK mapping. Performs single-flow deep analysis with chain-of-thought reasoning.

Why: IDS-Agent demonstrated that tool-augmented LLMs improve recall from 0.20 to 0.80 versus vanilla LLMs (OpenReview) (OpenReview) [9]. The eX-NIDS framework showed CTI-augmented prompts improve

explanation quality by 20%+ arXiv arXiv [2]. Critically, this agent receives the pre-filter's probability score as contextual evidence, anchoring the LLM's classification and mitigating the suspicious-by-default bias — the agent must argue against statistical evidence rather than classify from scratch.

Design detail: The agent receives a structured prompt containing only: (a) the flow's 53 features with human-readable labels, (b) the pre-filter's probability score and top contributing features (SHAP values), and (c) any threat intelligence from MCP tools. No temporal context, no calibration instructions — avoiding the prompt interference documented by the J6 framework [18] and context-as-noise effect [19].

Component 3: Temporal Correlation Agent (LLM)

What: A separate Claude-based agent that operates on batches of 10–20 flows grouped by source IP or temporal window, identifying multi-flow attack patterns (scanning, brute force, exfiltration sequences).

Why: Your Iteration 1 showed temporal memory improves benign accuracy (13.3% → 66.7%) but causes attack normalisation when combined with per-flow classification. The SagaLLM finding that "LLMs process each interaction independently, lacking native mechanisms to maintain state" VLDB Endowment Daily Dose of Data Science [22] means temporal reasoning must be architecturally separated. The Context Rot study confirmed temporal reasoning degrades under long contexts Trychroma [23]. A dedicated temporal agent receiving pre-batched flows avoids the single-agent ceiling you observed.

Design detail: This agent does not classify individual flows. It receives batches of pre-classified flows (from the pre-filter + Flow Analyst) and identifies patterns that should **upgrade** benign-classified flows to malicious (e.g., 50 SSH connection attempts from the same IP over 30 seconds, each individually classified as benign). It can also **downgrade** isolated malicious-classified flows that appear to be false positives in temporal context. This retroactive reclassification mechanism directly addresses the attack normalisation problem — the temporal agent only sees flow-level decisions and pattern evidence, not raw features.

Component 4: Orchestrator Agent

What: A lightweight coordinator that routes flows through the pipeline, resolves disagreements between agents, and maintains a shared memory store of session-level context.

Why: Quan et al. (2024) found that team structure critically impacts multi-agent performance, with hybrid configurations (centralised coordination + decentralised execution) outperforming purely centralised or decentralised approaches arXiv [38]. CrowdStrike Charlotte AI's router-agent pattern validates this at scale Exabeam [42]. Microsoft Security Copilot's support for serial, concurrent, and hybrid orchestration patterns Microsoft Learn demonstrates the need for flexible routing [43].

Design detail: The orchestrator implements a confidence-weighted voting mechanism when the Flow Analyst and Temporal Correlation agents disagree. Following Tran et al.'s (2025) finding that combining competition and cooperation outperforms either alone SaM Solutions [37], disagreements trigger a structured debate: each agent provides its classification with a confidence score and reasoning chain. The orchestrator applies a decision matrix:

- Both agents agree → accept classification
- Agents disagree, pre-filter probability >0.8 or <0.2 → defer to statistical evidence

- Agents disagree, pre-filter uncertain → escalate to human analyst with full reasoning from both agents

Component 5: Explainability Agent (LLM)

What: A final-stage agent that generates human-readable incident reports for all malicious-classified flows, incorporating MITRE ATT&CK mappings, temporal context, and recommended response actions.

Why: Houssel et al. (2024, 2025) established that LLMs' primary value in NIDS is explanation, not detection (arXiv) (arXiv) [1][2]. HuntGPT validated the separate-explainer pattern (ResearchGate +2) [3]. The "From Flows to Words" paper noted that LLM pipelines "produce human-readable artifacts" as their key advantage over ML baselines (arXiv) [8].

Design detail: This agent receives the final classification, the pre-filter's SHAP values, the Flow Analyst's reasoning chain, and any temporal patterns identified. It generates structured reports following the MITRE ATT&CK framework. This separation ensures explanation quality is not compromised by classification pressure.

Agent communication via MCP

All inter-agent communication uses MCP's tool and resource primitives, (AIMultiple) consistent with your existing 13-tool MCP infrastructure. The MCP specification (Anthropic, 2024) provides JSON-RPC 2.0 transport with standardised tool, resource, and prompt primitives (Model Context Protocol) [46]. AutoGen v0.4's native MCP support makes this architecturally straightforward (Microsoft) [44]. Each agent exposes its classification and reasoning as MCP resources that downstream agents can query. The shared memory store — containing IP reputation caches, session-level flow histories, and classification decisions — operates as an MCP server accessible to all agents. (Daily Dose of Data Science)

Dataset improvement: temporal batch construction

To properly evaluate the multi-agent architecture, the 58-flow stratified sample should be replaced with **temporally coherent batches** — sequences of 20–50 flows from the same time window, preserving the attack patterns that individual flow sampling destroys. Luay et al.'s (2025) V3 temporal features enable this (Uq) [31], but the batch construction must respect Engelen et al.'s (2021) documented flow construction errors (Distrinet-research) (Distrinet-research) [24]. Validating against UNSW-NB15 in parallel would establish cross-dataset generalisability (Taylor & Francis Online) [32].

Conclusion

Your experimental results are not anomalous — they represent the expected behaviour of single-agent LLM classification as documented across multiple independent studies. The **83.3% baseline F1 from flow features alone** is consistent with Houssel et al.'s finding that LLMs can extract meaningful signal from NetFlow features but cannot reliably calibrate their confidence. (arXiv) (arXiv) The **Iteration 3 collapse to 41.4% accuracy** is a textbook case of multi-objective prompt interference. The **0/142 meaningful external tool calls** reflects CICIDS2018's RFC 1918 addresses, not architectural failure.

The proposed five-agent architecture resolves each identified limitation through architectural separation rather than prompt engineering — a principle validated by CrowdStrike (12+ specialised agents, Exabeam 98%+ accuracy), CrowdStrike Microsoft Security Copilot (77+ agents), and academic frameworks like MALCDF and AgenticCyber. The key insight is that **the single-agent ceiling you demonstrated is not a limitation to be engineered around through better prompts, but a fundamental constraint that requires architectural decomposition**. Each agent does one thing well: statistical classification, uncertain-flow analysis, temporal pattern detection, coordination, or explanation. This directly addresses the recall-versus-benign-accuracy tradeoff that no single prompt modification could resolve.

IEEE reference list

[1] P. R. B. Houssel, P. Singh, S. Layeghy, and M. Portmann, "Towards Explainable Network Intrusion Detection using Large Language Models," *arXiv preprint arXiv:2408.04342*, 2024.

[2] P. R. B. Houssel, P. Singh, S. Layeghy, and M. Portmann, "eX-NIDS: A Framework for Explainable Network Intrusion Detection Leveraging Large Language Models," *arXiv preprint arXiv:2507.16241*, 2025.

[3] T. Ali and P. Kostakos, "HuntGPT: Integrating Machine Learning-Based Anomaly Detection and Explainable AI with Large Language Models (LLMs)," *arXiv preprint arXiv:2309.16021*, 2023.

[4] "XG-NID: Dual-Modality Network Intrusion Detection using a Heterogeneous Graph Neural Network and Large Language Model," *arXiv preprint arXiv:2408.16021*, 2024.

[5] M. Guastalla, Y. Li, A. Hekmati, and B. Krishnamachari, "Application of Large Language Models to DDoS Attack Detection," in *Security and Privacy in Cyber-Physical Systems and Smart Vehicles (SmartSP 2023)*, LNICST, vol. 552, Springer, 2024.

[6] D. Davey, K. Karim, H. Ragab Hassen, and H. Batatia, "Large Language Model-Based Network Intrusion Detection," in *Proc. Int. Conf. Information Technology and Applications (ICITA 2024)*, LNNS, vol. 1248, pp. 309–319, Springer, 2025.

[7] O. G. Lira, A. Marroquin, and M. A. To, "Harnessing the Advanced Capabilities of LLM for Adaptive Intrusion Detection Systems," in *Advanced Information Networking and Applications (AINA 2024)*, LNDECT, vol. 204, pp. 453–464, Springer, 2024.

[8] "From Flows to Words: Can Zero-/Few-Shot LLMs Detect Network Intrusions? A Grammar-Constrained, Calibrated Evaluation on UNSW-NB15," *arXiv preprint arXiv:2510.17883*, 2025.

[9] Y. Li, Z. Xiang, N. D. Bastian, D. Song, and B. Li, "IDS-Agent: An LLM Agent for Explainable Intrusion Detection in IoT Networks," in *NeurIPS 2024 Workshop on Open-World Agents*, 2024.

[10] M.-T. Bui *et al.*, "A Systematic Comparison of Large Language Models Performance for Intrusion Detection," *Proc. ACM Networking*, vol. 2, no. CoNEXT4, pp. 1–23, 2024, doi: 10.1145/3696379.

[11] F. Adjewa *et al.*, "LLM-based Continuous Intrusion Detection Framework for Next-Gen Networks," *arXiv preprint arXiv:2411.03354*, 2024.

[12] H. Zhang *et al.*, "When LLMs Meet Cybersecurity: A Systematic Literature Review," *Cybersecurity*, Springer, 2024, arXiv:2405.04760.

[13] M. A. Ferrag *et al.*, "Generative AI in Cybersecurity: A Comprehensive Review of LLM Applications and Vulnerabilities," *Future Generation Computer Systems*, vol. 173, p. 107877, 2025, arXiv:2405.12750.

[14] Y. Feng *et al.*, "Network Intrusion Detection: Evolution from Conventional Approaches to LLM Collaboration and Emerging Risks," *arXiv preprint arXiv:2510.23313*, 2025.

[15] "Bridging Expertise Gaps: The Role of LLMs in Human-AI Collaboration for Cybersecurity," *arXiv preprint arXiv:2505.03179*, 2025.

[16] "A crisis of overconfidence: Why confidence, not accuracy, is the real risk in clinical AI," *BioData Mining*, Springer Nature, 2026.

[17] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On Calibration of Modern Neural Networks," in *Proc. 34th Int. Conf. Machine Learning (ICML)*, PMLR, vol. 70, pp. 1321–1330, 2017.

[18] "J6: Jacobian-Driven Role Attribution for Multi-Objective Prompt Optimization in LLMs," *arXiv preprint arXiv:2508.12086*, 2025.

[19] "Is More Context Always Better? Examining LLM Reasoning," in *Proc. ACM Web Conf. (WWW '26)*, 2026, arXiv:2601.10132.

[20] "The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions," *arXiv preprint arXiv:2404.13208*, 2024.

[21] "Mitigating Overconfidence in Large Language Models: A Behavioral Lens on Confidence Estimation and Calibration," *OpenReview*, 2024.

[22] "SagaLLM: Context Management, Validation, and Transaction," in *Proc. VLDB Endowment*, vol. 18, pp. 4874ff, 2025.

[23] Chroma Research, "Context Rot: How Increasing Input Tokens Impacts LLM Performance," 2025. [Online]. Available: https://research.trychroma.com/context-rot

[24] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study," in *2021 IEEE Security and Privacy Workshops (SPW)*, pp. 7–12, 2021, doi: 10.1109/SPW53761.2021.00009.

[25] M. Lanvin, P.-F. Gimenez, Y. Han, F. Majorczyk, L. Mé, and E. Totel, "Errors in the CICIDS2017 Dataset and the Significant Differences in Detection Performances It Makes," in *Risks and Security of Internet and Systems (CRISIS)*, LNCS, Springer, 2023.

[26] L. Liu, G. Engelen, T. Lynar, D. Essam, and W. Joosen, "Error prevalence in NIDS datasets: A case study on CIC-IDS-2017 and CSE-CIC-IDS-2018," in *IEEE Conf. Communications and Network Security (CNS)*, pp. 254–262, 2022.

[27] R. Panigrahi and S. Borah, "A Detailed Analysis of CICIDS2017 Dataset for Designing Intrusion Detection Systems," *Int. J. Engineering and Technology*, vol. 7, no. 3.24, pp. 479–482, 2018.

[28] J. L. Leevy and T. M. Khoshgoftaar, "A Survey and Analysis of Intrusion Detection Models Based on CSE-CIC-IDS2018 Big Data," *Journal of Big Data*, vol. 7, Springer, 2020.

[29] "Transferability of Machine Learning Models Learned from Public Intrusion Detection Datasets: The CICIDS2017 Case Study," *Software Quality Journal*, Springer, 2022.

[30] A. Kenyon, L. Deka, and D. A. Elizondo, "Are Public Intrusion Datasets Fit for Purpose: Characterising the State of the Art in Intrusion Event Datasets," *Computers & Security*, vol. 99, p. 102022, 2020.

[31] M. Luay, S. Layeghy, S. Hosseininoorbin, M. Sarhan, N. Moustafa, and M. Portmann, "Temporal Analysis of NetFlow Datasets for Network Intrusion Detection Systems," *arXiv preprint arXiv:2503.04404*, 2025.

[32] N. Moustafa and J. Slay, "UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems," in *Military Communications and Information Systems Conf. (MilCIS)*, IEEE, 2015.

[33] "MALCDF: A Distributed Multi-Agent LLM Framework for Real-Time Cyber Defense," *arXiv preprint arXiv:2512.14846*, 2025.

[34] "A Multi-Agent System for Cybersecurity Threat Detection and Correlation Using Large Language Models," *IEEE Journals & Magazine*, 2025, doi: 10.1109/11141466.

[35] "AgenticCyber: A GenAI-Powered Multi-Agent System for Multimodal Threat Detection," *arXiv preprint arXiv:2512.06396*, 2025.

[36] "Audit-LLM: Multi-Agent Collaboration for Log-based Insider Threat Detection," *arXiv preprint arXiv:2408.08902*, 2024.

[37] Tran *et al.*, "Multi-Agent Collaboration Mechanisms: A Survey of LLMs," *arXiv preprint arXiv:2501.06322*, 2025.

[38] Y. Quan *et al.*, "Multi-Agent Collaboration in Incident Response with Large Language Models," *arXiv preprint arXiv:2412.00652*, 2024.

[39] T. Guo *et al.*, "Large Language Model based Multi-Agents: A Survey of Progress and Challenges," in *Proc. IJCAI 2024*, pp. 8048–8057, 2024.

[40] X. Li, S. Wang, S. Zeng, Y. Wu, and Y. Yang, "A Survey on LLM-based Multi-Agent Systems: Workflow, Infrastructure, and Challenges," *Vicinagearth*, vol. 1, no. 1, p. 9, Springer Nature, 2024.

[41] Y. Talebirad and A. Nadiri, "Multi-Agent Collaboration: Harnessing the Power of Intelligent LLM Agents," *arXiv preprint arXiv:2306.03314*, 2023.

[42] CrowdStrike, "Charlotte AI Multi-Approach Architecture," 2024. [Online]. Available: https://www.crowdstrike.com/en-us/blog/charlotte-ai-multi-approach/

[43] Microsoft, "Multi-Agent Patterns in Microsoft Copilot Studio," 2025. [Online]. Available: https://learn.microsoft.com/en-us/microsoft-copilot-studio/guidance/architecture/multi-agent-patterns

[44] Q. Wu *et al.*, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," *arXiv preprint arXiv:2308.08155*, 2023.

[45] M. A. Ferrag *et al*., "Revolutionizing Cyber Threat Detection with Large Language Models: A privacy-preserving BERT-based Lightweight Model for IoT/IIoT Devices," *IEEE Access*, vol. 12, pp. 18424–18441, 2024, arXiv:2306.14263.

[46] Anthropic, "Model Context Protocol Specification," Nov. 2024. [Online]. Available: https://modelcontextprotocol.io/specification/2025-11-25

[47] "The Paradigm of Hallucinations in AI-driven Cybersecurity Systems: Understanding Taxonomy, Classification Outcomes, and Mitigations," *Computers and Electrical Engineering*, Elsevier, 2025.

[48] M. Sarhan, S. Layeghy, N. Moustafa, and M. Portmann, "NetFlow Datasets for Machine Learning-Based Network Intrusion Detection Systems," in *Big Data Technologies and Applications (BDTA 2020)*, LNICST, vol. 371, pp. 117–135, Springer, 2021.

[49] M. Sarhan, S. Layeghy, and M. Portmann, "Towards a Standard Feature Set for Network Intrusion Detection System Datasets," *Mobile Networks and Applications*, vol. 27, pp. 357–370, 2022.

[50] "A Survey of Agentic AI and Cybersecurity: Challenges, Opportunities and Use-case Prototypes," *arXiv preprint arXiv:2601.05293*, 2025.

[51] S. Yao *et al*., "ReAct: Synergizing Reasoning and Acting in Language Models," *arXiv preprint arXiv:2210.03629*, 2022.

[52] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in *4th Int. Conf. Information Systems Security and Privacy (ICISSP)*, 2018.